Bare metal and datacenter

Rack, provision and operate physical servers: BMC and IPMI, PXE provisioning, disks and RAID, leaf-spine networking and fleet operations.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Servers, racks and remote management (13 min)
2. Provisioning: PXE, DHCP and images (14 min)
3. Disks, RAID, firmware and burn-in (14 min)
4. Datacenter networking: leaf-spine and bonding (14 min)
5. Operating a fleet: failures, lifecycle and capacity (15 min)
6. BMC, IPMI and Redfish automation (12 min)
7. UEFI, bootloaders and Secure Boot (12 min)
8. cloud-init and first-boot configuration (12 min)
9. Firmware and BIOS management at scale (12 min)
10. MAAS and Ironic: provisioning bare metal as a service (13 min)
11. Hardware failure diagnosis and RMA (12 min)
12. Power, cooling and rack capacity planning (12 min)
Cheat sheet

Lesson 1. Servers, racks and remote management

13 minutes. Learn what is inside a rack server, how racks are powered and cooled, and how to manage a machine from far away through its BMC.

A virtual machine is a slice of someone else's server. On bare metal you own the whole machine, and you also own everything that surrounds it: the rack, the power, the cooling and the cable to the switch. This lesson shows the parts and the one tool that lets you manage them without being in the room.

What you will learn

- The parts of a rack server and how racks are laid out
- Power, cooling and why a rack has a limit
- The BMC: the small computer that manages the big one
- IPMI and Redfish, and the commands you use every week

What is in the box

A rack server is measured in **rack units**: 1U is 4.45 cm tall. Most fleets use 1U or 2U servers. Inside you find:

- **CPUs** and **DIMMs** (memory sticks), usually in pairs per channel
- **Drives**: SATA, SAS or NVMe, in front bays or inside
- **Network cards** (NICs), often two or four ports
- **Power supplies**, normally two, each fed from a different power line
- **Fans**, which pull air from the front (cold aisle) to the back (hot aisle)
- A **BMC**, the baseboard management controller, a tiny separate computer on the motherboard

Power and cooling set the limit

A rack is not full when it runs out of space. It is full when it runs out of **power** or **cooling**. A rack with a 7 kW feed cannot hold twenty servers that each draw 500 W, even though they fit.

- Two power supplies on two separate feeds (A and B) mean one feed can fail and the server stays up, **but only if each feed alone can carry the whole rack**.
- Cold air enters at the front. Blanking panels in empty slots stop hot air from looping back round.
- A hot spot at the top of a rack is usually a gap in the panels or a failed fan.

The BMC: a computer inside the computer

The BMC has its own network port, its own address and its own power, so it works when the main server is off or hung. You talk to it with **IPMI** (older, widely supported) or **Redfish** (newer, a REST API with JSON). Vendors brand it as iDRAC, iLO or IPMI/BMC.

With it you can power the server on or off, read the hardware event log, check temperatures and open a remote console, all without going near the machine.

```
$ ipmitool -H 10.20.0.50 -U admin -P secret chassis power status
Chassis Power is off
$ ipmitool -H 10.20.0.50 -U admin -P secret sel list
 1   | 11/02/2026 | 03:14:22 | Power Unit #0x01 | Power off/down | Asserted
 2   | 11/02/2026 | 03:14:25 | Temperature #0x31 | Upper Critical going high | Asserted
$ ipmitool -H 10.20.0.50 -U admin -P secret chassis power on
Chassis Power Control: Up/On
```

MARGA by EknathaLabs marga.eknathalabs.com

The **system event log** (SEL) is the first place to look. Here the server shut itself down because it overheated, which points at cooling and not at software.

Keep the BMC safe

The BMC can power-cycle a server and open a console, so treat its network as a management network: separate from production, not reachable from the internet, with its default password changed on day one.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Servers, racks and remote management fits together

Part	What it does	What goes wrong
What is in the box	A rack server is measured in rack units: 1U is 4.45 cm tall.	Filling a rack by space and ignoring power and cooling.
Power and cooling set the limit	A rack is not full when it runs out of space.	Plugging both power supplies into the same feed.
The BMC: a computer inside the	The BMC has its own network port, its own address and its own power, so it works when the main server is off or hung.	Leaving the BMC on the production network with the default password.
Keep the BMC safe	The BMC can power-cycle a server and open a console, so treat its network as a management network: separate from production, not reachable from the internet, with its...	Having no way in but SSH, so a hung host needs a trip to the datacentre.

Flow: What is in the box to Power and cooling set the limit / Power and cooling set the limit to The BMC: a computer inside the / The BMC: a computer inside the to Keep the BMC safe

Incident: The rack that tripped at noon

Incident: *Four servers in one rack went off within a minute at 12:04.*

```
$ ipmitool -H 10.20.0.61 sel list | tail -2
 9   | 11/02/2026 | 12:04:11 | Power Supply #0x62 | Power Supply AC lost | Asserted
10   | 11/02/2026 | 12:04:12 | Power Unit #0x01 | Redundancy Lost | Asserted
```

One power feed failed, as the design allows. But the extra servers added over the year had pushed the load so that feed B alone could not carry the rack. When A dropped, B tripped too. The fix was to add up the load on one feed and to stop adding servers to a rack once one feed cannot carry it alone.

Common mistakes

- Filling a rack by space and ignoring power and cooling
- Leaving the BMC on the production network with the default password
- Plugging both power supplies into the same feed
- Having no way in but SSH, so a hung host needs a trip to the datacentre

Check yourself

What is the BMC?

Answer: A small separate computer on the motherboard that manages the server remotely

A rack has space for 20 servers but one feed can carry only 7 kW. Each server draws 500 W. How many can it safely hold if one feed must carry the whole rack?

Answer: 14

Which log is the first place to look when a server shut itself off?

Answer: The BMC system event log (SEL)

MARGA by EknathaLabs marga.eknathalabs.com

Why do servers have two power supplies on separate feeds?

Answer: So one feed or supply can fail without the server stopping

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Provisioning: PXE, DHCP and images

14 minutes. Follow a new server from power-on to a running operating system with PXE, DHCP, TFTP and an automated install, and learn why each step can fail.

A new server arrives with empty disks. Nobody wants to install an operating system on a hundred machines by hand. Provisioning means the machine installs itself over the network, the same way every time.

What you will learn

- The boot chain: DHCP, TFTP or HTTP, a boot loader, an installer
- What a kickstart, preseed or cloud-init file does
- Image based provisioning compared with installing from packages
- Tools that manage the process: MAAS, Ironic and Foreman

The boot chain

1. The server powers on and its network card sends a **DHCP** request. The BIOS or UEFI was set to try the network first.
2. The DHCP server answers with an IP address, plus two extra pieces of information: `next-server` (where the boot files are) and `filename` (which file to load).
3. The server fetches the boot loader (for example `pxelinux.0` or `grubx64.efi`) over **TFTP**. Modern setups chain to **iPXE**, which can use HTTP and is much faster.
4. The boot loader loads a kernel and an initial ramdisk and starts an installer.
5. The installer reads an **answer file** and installs without asking questions, then reboots from disk.

```
$ sudo journalctl -u dnsmasq -n 4
dnsmasq-dhcp: DHCPDISCOVER(eth0) 3c:ec:ef:12:34:56
dnsmasq-dhcp: DHCPOFFER(eth0) 10.20.1.77 3c:ec:ef:12:34:56
dnsmasq-tftp: sent /srv/tftp/grubx64.efi to 10.20.1.77
dnsmasq-tftp: file /srv/tftp/grub.cfg-3c:ec:ef:12:34:56 not found
```

The last line is normal: the loader first looks for a config file named after its own MAC address, then falls back to a default.

The answer file

- **Kickstart** (Red Hat family), **preseed** (Debian) and **autoinstall** (Ubuntu) describe disks, packages, users and network.
- **cloud-init** runs on first boot and sets the hostname, keys and extra setup.

A good rule: the installer makes the machine generic, and configuration management (Ansible, Puppet) makes it specific. That way, rebuilding is safe and repeatable.

Images instead of installs

Installing packages on every machine is slow and each one drifts. **Image based** provisioning writes a prepared disk image to the machine, then lets cloud-init personalise it. It is faster and every node starts identical. OpenStack Ironic and MAAS both work this way.

Tools

- **MAAS** (Canonical) discovers machines, controls power through the BMC and deploys images.
- **OpenStack Ironic** makes bare metal servers look like cloud instances.
- **Foreman** with Katello manages Red Hat style installs and patching.

MARGA by EknathaLabs
marga.eknathalabs.com

All of them rest on the same chain, so when one fails you debug DHCP, then TFTP or HTTP, then the loader, then the installer.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Provisioning: PXE, DHCP and images fits together

Part	What it does	What goes wrong
The boot chain	The server powers on and its network card sends a DHCP request.	Leaving network boot first on machines in service.
The answer file	Kickstart (Red Hat family), preseed (Debian) and autoinstall (Ubuntu) describe disks, packages, users and network.	Putting settings inside the install image instead of in cloud-init or configuration management.
Images instead of installs	Installing packages on every machine is slow and each one drifts.	Running a single provisioning server that is itself unavailable when you need it.
Tools	MAAS (Canonical) discovers machines, controls power through the BMC and deploys images.	Having no DHCP log to read when PXE fails.

Flow: The boot chain to The answer file / The answer file to Images instead of installs / Images instead of installs to Tools

Incident: Every new node booted into a reinstall

> **Incident:** *After an upgrade, nodes that were already in service rebooted into the installer and wiped themselves.*

The boot order on those servers was network first. Normally the PXE server answered only unknown machines. After the change, it answered every machine, so a routine reboot ran the installer again. The fix was to answer only machines marked "to be built", and to set the disk first in the boot order once a machine is deployed.

Common mistakes

- Leaving network boot first on machines in service
- Putting settings inside the install image instead of in cloud-init or configuration management
- Having no DHCP log to read when PXE fails
- Running a single provisioning server that is itself unavailable when you need it

Check yourself

Which two DHCP options tell a PXE client where to boot from?

Answer: next-server and the boot file name

Why is image based provisioning popular?

Answer: It is faster and every node starts identical

PXE shows an offer in the DHCP log but no TFTP transfer. What do you check?

Answer: The next-server address and boot file name

What should a deployed server's boot order be?

Answer: Disk first, with network boot only for rebuilds

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Disks, RAID, firmware and burn-in

14 minutes. Understand drive types, software and hardware RAID, SMART health, firmware and the burn-in tests that catch bad hardware before it carries data.

Disks are the part of a server that wears out. On bare metal you choose them, protect them and replace them, so you need to know how they fail.

What you will learn

- SATA, SAS and NVMe, and what each costs and delivers
- RAID levels and the difference between software and hardware RAID
- Reading SMART data and acting on it
- Firmware and BIOS settings that matter
- Burn-in: testing a new machine before it carries load

Drive types

- **SATA** is cheap and fine for capacity. **SAS** is built for servers: dual path, better error handling.
- **NVMe** connects directly over PCIe. It is far faster and handles many requests at once, which suits databases.
- Drives are rated for **endurance** (how many times you can rewrite the whole disk per day). A write-heavy database on a read-optimised drive wears it out early.

RAID in short

Level	What it does	Survives
0	Stripes data for speed	Nothing
1	Mirrors two disks	One disk
5	Stripes with one parity	One disk
6	Stripes with two parity	Two disks
10	Mirrored pairs, striped	One per pair

RAID protects against a **disk failing**. It does not protect against deleting a file, a bad application update, or a fire. You still need backups.

Linux can do RAID in software with `mdadm`. Hardware RAID cards do it in firmware. Software RAID is easier to inspect and move between machines; hardware RAID often has a battery-backed write cache that helps write speed.

```
$ sudo mdadm --detail /dev/md0 | grep -E "State|Failed|Active"
            State : clean, degraded
    Active Devices : 1
    Failed Devices : 1
```

Degraded means the array works but has lost its safety margin. Until the replacement finishes rebuilding, a second failure loses data.

SMART: the disk's own report

Disks monitor themselves. `smartctl` reads that report.

```
$ sudo smartctl -H /dev/sdb
SMART overall-health self-assessment test result: FAILED!
$ sudo smartctl -a /dev/sdb | grep -E "Reallocated|Pending|Power_On"
  5 Reallocated_Sector_Ct   098   098   010    120
  9 Power_On_Hours          090   090   000    43000
197 Current_Pending_Sector  100   100   000    8
```

MARGA by EknathaLabs
marga.eknathalabs.com

Rising **reallocated** or **pending** sectors mean the disk is running out of spare space. Replace it before it fails completely.

Firmware and BIOS

Disks, NICs, the BMC and the BIOS all have firmware. Out-of-date firmware causes strange bugs and sometimes data loss. Update in a controlled order, one node at a time, and keep a record. Check BIOS settings such as boot mode (UEFI), power profile (performance, not power saving, for latency-sensitive work) and virtualisation support.

Burn-in

New hardware fails early more often than old hardware. A **burn-in** run for a day or two catches it before real data lands there:

- Memory test (`memtester` or the vendor tool)
- CPU stress (`stress-ng`)
- Disk write and read of the whole surface (`fio` or `badblocks`)
- Check the event log and SMART afterwards for any new errors

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Disks, RAID, firmware and burn-in fits together

Part	What it does	What goes wrong
Drive types	SATA is cheap and fine for capacity. SAS is built for servers: dual path, better error handling.	Updating firmware on every node at once.
RAID in short	RAID protects against a disk failing. It does not protect against deleting a file, a bad application update, or a fire.	Ignoring a degraded array for days.
SMART: the disk's own report	Disks monitor themselves. smartctl reads that report.	Replace it before it fails completely.
Firmware and BIOS	Disks, NICs, the BMC and the BIOS all have firmware.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Burn-in	New hardware fails early more often than old hardware.	Skipping burn-in and finding bad memory in production.

Flow: Drive types to RAID in short / RAID in short to SMART: the disk's own report / SMART: the disk's own report to Firmware and BIOS / Firmware and BIOS to Burn-in

Incident: Two disks, one batch

Incident: *A RAID 5 array lost a second disk during the rebuild after the first one failed.*

All the disks came from the same batch, were installed on the same day and had the same hours on them. When the first failed, the rebuild put heavy load on the others, and the second failed within hours. After this the team used RAID 6 for large arrays, mixed batches, and made rebuild time part of their capacity plan.

Common mistakes

- Treating RAID as a backup
- Ignoring a degraded array for days
- Skipping burn-in and finding bad memory in production
- Updating firmware on every node at once

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does RAID 1 do?

Answer: Mirrors data across two disks, surviving one failure

An array shows State: clean, degraded. What does that mean?

Answer: It works but has lost its redundancy until the replacement rebuilds

Which SMART values suggest a disk is dying?

Answer: Rising reallocated and pending sector counts

Why run a burn-in on new hardware?

Answer: New hardware fails early more often, so test before real data arrives

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Datacenter networking: leaf-spine and bonding

14 minutes. See how racks connect with top-of-rack switches in a leaf-spine fabric, and how bonding, VLANs and a separate management network keep servers reachable.

A server is only useful when it can reach other servers. In a datacentre, the way racks connect decides how fast and how reliably they can talk.

What you will learn

- Top-of-rack switches and the leaf-spine layout
- Why east-west traffic changed the design
- Bonding and LACP for redundant links
- VLANs and the separate management network
- Cabling mistakes that cause hard-to-find faults

Top-of-rack and leaf-spine

Each rack has a **top-of-rack (ToR) switch**, which connects the servers in that rack. In a **leaf-spine** design the ToR switches are the **leaves**, and every leaf connects to every **spine** switch.

- Any two servers are at most three switch hops apart (leaf, spine, leaf).
- More capacity means adding spines. More racks mean adding leaves.
- Traffic is spread over many equal-cost paths with **ECMP**.

This suits modern workloads, where most traffic is **east-west** (server to server inside the datacentre) and not north-south (in and out).

Bonding and LACP

A server with two NIC ports can join them into one logical link, called a **bond** (Linux) or a team. This gives redundancy and, with the right mode, more bandwidth.

Mode	What it does	Switch support needed
active-backup	One link used, the other waits	None
802.3ad (LACP)	Both links used, negotiated	Switch must run LACP
balance-rr	Packets spread round-robin	Often causes reordering

LACP needs both sides to agree. If the server says 802.3ad but the switch ports are not in a port channel, the link will not form properly.

```
$ cat /proc/net/bonding/bond0 | grep -E "Mode|Slave Interface|MII Status"
Bonding Mode: IEEE 802.3ad Dynamic link aggregation
MII Status: up
Slave Interface: eno1
MII Status: up
Slave Interface: eno2
MII Status: down
```

One slave is down: either the cable, the optic or the switch port is at fault.

VLANs and the management network

A **VLAN** splits one physical network into separate logical ones. A common set: production, storage, a provisioning network for PXE, and a **management** network for the BMCs. Keep management traffic apart from production traffic so a problem or an attacker on one cannot reach the other.

MARGA by EknathaLabs marga.eknathalabs.com

Cabling

Many faults are physical: a bent fibre, a dirty connector, a wrong optic, a cable plugged into the wrong port. Label both ends and keep a record of which server port goes to which switch port. Check counters with `ip -s link` for errors and drops.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Datacenter networking: leaf-spine and bonding fits together

Part	What it does	What goes wrong
Top-of-rack and leaf-spine	Each rack has a top-of-rack (ToR) switch, which connects the servers in that rack.	Putting management and production traffic on one network.
Bonding and LACP	A server with two NIC ports can join them into one logical link, called a bond (Linux) or a team.	Hiding link flaps because the bond keeps traffic flowing.
VLANs and the management network	A VLAN splits one physical network into separate logical ones.	Keep management traffic apart from production traffic so a problem or an attacker on one cannot reach the other.
Cabling	Many faults are physical: a bent fibre, a dirty connector, a wrong optic, a cable plugged into the wrong port.	Configuring LACP on only one side.

Flow: Top-of-rack and leaf-spine to Bonding and LACP / Bonding and LACP to VLANs and the management network / VLANs and the management network to Cabling

Incident: A server that dropped off every ten minutes

Incident: *A server lost network for a few seconds, over and over.*

```
$ dmesg | grep -i bond | tail -3
bond0: link status definitely down for interface eno2, disabling it
bond0: link status definitely up for interface eno2, 10000 Mbps full duplex
```

The cable was seated badly in one port, so the link bounced. The bond kept the server up, but the switch logged flapping. Replacing the cable fixed it. The lesson: bonding hides physical faults, so alert on link changes, not only on the server being down.

Common mistakes

- Configuring LACP on only one side
- Putting management and production traffic on one network
- Not labelling cables
- Hiding link flaps because the bond keeps traffic flowing

Check yourself

In a leaf-spine fabric, how many switch hops are there between servers in different racks?

Answer: Three: leaf, spine, leaf

Which bonding mode needs the switch to be configured to match?

Answer: 802.3ad (LACP)

Why keep a separate management network?

Answer: So BMCs and admin access are isolated from production traffic and attackers

A bond keeps a server online while a cable flaps. Why can that be a problem?

Answer: It hides the physical fault unless you alert on link changes

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Operating a fleet: failures, lifecycle and capacity

15 minutes. Diagnose common hardware failures, plan the life of a server from rack to retirement, and keep enough spare capacity and parts.

With hundreds of servers, hardware failure is routine, not exceptional. Operating bare metal well means finding faults quickly, replacing parts safely and planning ahead.

What you will learn

- Reading hardware events: ECC memory, disks, fans, power
- A repeatable method for a sick server
- The lifecycle: receive, rack, provision, run, repair, retire
- Spares, warranty and capacity
- How bare metal and the cloud differ in who fixes what

Common hardware failures

Sign	Likely cause	First check
Random reboots	Failing memory or power supply	BMC event log
Correctable ECC errors	A DIMM starting to fail	Which slot, how often
Slow disk, many retries	Disk with bad sectors	`smartctl -a`
Fan or temperature alarms	Failed fan, blocked airflow	`ipmitool sensor list`
Network errors	Bad cable, optic or port	`ip -s link`, switch counters

ECC memory corrects single-bit errors silently. A few correctable errors are normal. A DIMM that keeps reporting them is likely to produce an **uncorrectable** error soon, which crashes the machine. Replace it at the first maintenance window.

```
$ ipmitool sel elist | grep -i ecc
  3 | 11/02/2026 | 03:14:22 | Memory #0x87 | Correctable ECC | Asserted
  4 | 11/02/2026 | 03:41:09 | Memory #0x87 | Correctable ECC | Asserted
  5 | 11/02/2026 | 04:02:55 | Memory #0x87 | Correctable ECC | Asserted
```

The same sensor three times in an hour points at one DIMM slot.

A method for a sick server

1. **Drain** it: move workloads away so repair is safe.
2. **Read** the evidence: BMC log, SMART, kernel log, switch counters.
3. **Decide** the part: do not swap parts at random.
4. **Replace** one thing, then test.
5. **Record** it, so a repeat failure on the same machine shows up.
6. Return it to service only after it passes a short burn-in.

Lifecycle

A server goes through receiving (check against the order and asset tag), racking, provisioning, service, repair and retirement. Retirement includes **wiping disks securely** and removing the machine from inventory, monitoring, DNS and the configuration database. Forgotten machines keep drawing power and keep old credentials alive.

Spares and warranty

MARGA by EknathaLabs marga.eknathalabs.com

- Keep spare drives, DIMMs, power supplies and cables on site, since shipping can take days.
- Know the warranty terms. Vendors usually ship the part but you replace it, unless you buy on-site service.
- Track **mean time between failures** across your fleet. If one batch fails faster, ask the vendor early.

Capacity

Plan for losing a rack and a feed. Leave room so that when machines are out for repair, the rest still carry the load. Order lead times for hardware can be weeks or months, so plan quarters ahead, not days.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Operating a fleet: failures, lifecycle and... fits together

Part	What it does	What goes wrong
Common hardware failures	ECC memory corrects single-bit errors silently.	Replacing parts at random.
A method for a sick server	Drain it: move workloads away so repair is safe.	Decide the part: do not swap parts at random.
Lifecycle	A server goes through receiving (check against the order and asset tag), racking, provisioning, service, repair and retirement.	Forgetting to wipe disks and remove retired servers from inventory.
Spares and warranty	Keep spare drives, DIMMs, power supplies and cables on site, since shipping can take days.	Having no spares and waiting days for a cable.
Capacity	Plan for losing a rack and a feed. Leave room so that when machines are out for repair, the rest still carry the load.	Updating firmware everywhere at once.

Flow: Common hardware failures to A method for a sick server / A method for a sick server to Lifecycle / Lifecycle to Spares and warranty / Spares and warranty to Capacity

Incident: A fleet-wide reboot loop after a firmware update

Incident: *After a firmware update across 120 servers, 30 would not come back.*

The update was pushed to every host at once. A batch with an older BMC version needed an intermediate firmware first, and without it the BMC hung. With no BMC, the team had to visit the datacentre. Afterwards updates went in waves of five, starting with one canary, with a check that each host was healthy before the next wave.

Common mistakes

- Replacing parts at random
- Updating firmware everywhere at once
- Forgetting to wipe disks and remove retired servers from inventory
- Having no spares and waiting days for a cable

Check yourself

A DIMM reports correctable ECC errors several times in an hour. What should you do?

Answer: Plan to replace that DIMM soon, since it is likely to fail

What should you do first with a failing server that carries workloads?

Answer: Drain it so the repair is safe

Why roll out firmware in small waves?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: So a bad update affects a few servers, not the whole fleet

What does retiring a server involve besides unracking it?

Answer: Wiping its disks and removing it from inventory, monitoring and DNS

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. BMC, IPMI and Redfish automation

12 minutes. Control servers out of band with the BMC, use IPMI and Redfish safely, and automate power, boot and sensor tasks at scale.

When a server hangs or has no operating system, you still need to reach it. The baseboard management controller, or BMC, is a separate chip with its own network port that gives you power control, console and hardware health without touching the host. Redfish is the modern, secure API on top of it.

What you will learn

- What the BMC is and why it works when the host is off
- IPMI versus Redfish and their security differences
- Scripting power, boot device and sensor reads
- Keeping the management network safe

IPMI and Redfish

IPMI is the older protocol. It is widely supported but has weak authentication and many known issues, so use it only on a locked management network. **Redfish** is a REST and JSON API over HTTPS that covers inventory, power, boot, firmware and sensors with modern authentication.

```
# IPMI over the LAN
ipmitool -I lanplus -H 10.20.0.15 -U admin -P "$BMC_PASS" chassis power status
ipmitool -I lanplus -H 10.20.0.15 -U admin -P "$BMC_PASS" chassis bootdev pxe options=efiboot

# Redfish: read power state and reset the system
curl -sk -u "admin:$BMC_PASS" https://10.20.0.15/redfish/v1/Systems/1 | jq '.PowerState,.Status'
curl -sk -u "admin:$BMC_PASS" -X POST -H 'Content-Type: application/json' \
  -d '{"ResetType":"GracefulRestart"}' \
  https://10.20.0.15/redfish/v1/Systems/1/Actions/ComputerSystem.Reset
```

Automating a fleet

Keep an inventory that maps each server to its BMC address, and fetch credentials from a secrets manager instead of a file. Wrap actions in a small script that does one server at a time, logs who did what and has a dry-run mode. Common jobs are rebooting a hung node, setting one-time PXE boot before reinstall, reading temperature and fan sensors, and collecting the system event log when a node fails.

Keeping it safe

Put every BMC on a separate **management network** with firewall rules that allow only a jump host. Change default accounts, disable unused services such as old IPMI cipher suites, update BMC firmware on a schedule and send BMC logs to your central logging system. A BMC compromise survives operating system reinstalls, so prevention matters.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How BMC, IPMI and Redfish automation fits together

Part	What it does	What goes wrong
IPMI and Redfish	IPMI is the older protocol. It is widely supported but has weak authentication and many known issues, so use it only on a locked management network.	Scripts that reboot many servers at once without a limit.
Automating a fleet	Keep an inventory that maps each server to its BMC address, and fetch credentials from a secrets manager instead of a file.	Leaving default BMC credentials in place.
Keeping it safe	Put every BMC on a separate management network with firewall rules that allow only a jump host.	Putting BMCs on the production network.

Flow: IPMI and Redfish to Automating a fleet / Automating a fleet to Keeping it safe

Incident: The open management network

Incident: *Many BMCs were reachable on the office network with default logins.*

```
$ nmap -p 623,443 10.20.0.0/24 --open
10.20.0.15  623/udp open  asf-rmcp
10.20.0.15  login admin:admin accepted
```

An audit found that BMCs were on the same network as laptops and still used factory credentials. Anyone on the network could power machines off or mount a virtual disk. The team moved all BMCs to an isolated VLAN behind a jump host, rotated unique passwords from a vault and added alerts for new BMC logins.

Common mistakes

- Leaving default BMC credentials in place
- Putting BMCs on the production network
- Never updating BMC firmware
- Scripts that reboot many servers at once without a limit

Check yourself

What does the BMC allow when the host is off?

Answer: Remote power, console and sensors

Which is the modern API for BMC control?

Answer: Redfish

Where should BMCs live?

Answer: A isolated management network

Why add a dry-run mode to fleet scripts?

Answer: To avoid accidental mass reboots

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. UEFI, bootloaders and Secure Boot

12 minutes. Follow the boot path from firmware to kernel, manage UEFI boot entries and understand Secure Boot keys and signed bootloaders.

A server is only as trustworthy as the first code it runs. Modern machines use UEFI firmware, an EFI system partition and a chain of signed bootloaders. Knowing this chain makes boot failures and security questions much less mysterious.

What you will learn

- The UEFI boot sequence from firmware to kernel
- Reading and changing boot entries with efibootmgr
- How Secure Boot verifies shim, bootloader and kernel
- Recovering from a machine that will not boot

The chain

Firmware reads **boot entries** from NVRAM and starts the first valid one. That loads a bootloader such as GRUB from the **EFI system partition** (ESP), a small FAT partition mounted at `/boot/efi`. The bootloader loads the kernel and initramfs, and the kernel mounts the root filesystem.

```
efibootmgr -v
mount | grep efi
ls /boot/efi/EFI
```

```
BootCurrent: 0003
BootOrder: 0003,0001,0000
Boot0003* ubuntu  HD(1,GPT,...)/File(\EFI\ubuntu\shimx64.efi)
Boot0001* UEFI PXE IPv4 Intel(R) Ethernet
```

Change the order with `efibootmgr -o 0003,0001` and set a one-time next boot with `efibootmgr -n 0001`. Always keep a working entry before removing another.

Secure Boot

With **Secure Boot**, firmware only runs code signed by a key it trusts. Distributions use a small signed program called **shim**, which verifies GRUB and the kernel against the vendor key or a Machine Owner Key you enroll. Unsigned custom kernels or out-of-tree modules fail unless you sign them.

```
mokutil --sb-state
mokutil --list-enrolled | head
```

Recovery

If a node will not boot, use the BMC console to see the error. Boot a rescue image, mount the root and ESP, then reinstall the bootloader and regenerate configuration. Keep a tested rescue image on your provisioning server so fixing a node is routine.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How UEFI, bootloaders and Secure Boot fits together

Part	What it does	What goes wrong
The chain	Firmware reads boot entries from NVRAM and starts the first valid one.	Removing the old boot entry before testing the new one.
Secure Boot	With Secure Boot, firmware only runs code signed by a key it trusts.	Disabling Secure Boot instead of signing correctly.
Recovery	If a node will not boot, use the BMC console to see the error.	Check the Common mistakes list below this diagram for what usually goes wrong here.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: The chain to Secure Boot / Secure Boot to Recovery

Incident: The kernel that would not start

Incident: *A custom kernel was rejected after Secure Boot was enabled.*

```
error: bad shim signature
error: you need to load the kernel first
Press any key to continue
```

A fleet moved to Secure Boot by policy, but the in-house kernel was unsigned. Nodes stopped at the loader. The team signed the kernel with an enrolled key, kept the signing key in a hardware module and added a boot test on one canary node before every rollout.

Common mistakes

- Updating firmware on the whole fleet at once
- Removing the old boot entry before testing the new one
- Disabling Secure Boot instead of signing correctly
- Not backing up the ESP contents

Check yourself

What does the ESP hold?

Answer: Bootloader files

What is shim?

Answer: A small signed loader that verifies the next stage

Which command lists UEFI boot entries?

Answer: efibootmgr -v

What is the safest way to roll out firmware updates?

Answer: Canary first, then waves

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. cloud-init and first-boot configuration

12 minutes. Configure a new machine on its first boot with cloud-init user data, and debug why a node came up without its settings.

Building a new image for every server does not scale. Instead you build one image and let cloud-init configure each machine the first time it boots, using data supplied by the platform, a metadata service or a small seed disk.

What you will learn

- Datasources, metadata and user data
- Writing a cloud-config file
- The stages cloud-init runs through
- Debugging a first boot that did not apply

cloud-config

```
#cloud-config
hostname: web-014
users:
  - name: ops
    groups: [sudo]
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-ed25519 AAAAC3Nza... ops@example.com
package_update: true
packages:
  - chrony
  - nginx
write_files:
  - path: /etc/motd
    content: |
      Managed by platform team
runcmd:
  - [systemctl, enable, --now, chrony]
```

The file starts with `#cloud-config` and uses modules such as users, packages, write_files and runcmd. It is handed to the machine as **user data**, while **metadata** such as instance ID and hostname comes from the datasource: a cloud metadata service, a config drive or a NoCloud seed.

Stages

cloud-init runs in order: **local** (find the datasource), **network** (get metadata and user data), **config** (run configuration modules) and **final** (package installs and scripts). Most per-instance work runs only once, keyed by instance ID, so rebooting does not repeat it.

Debugging

```
cloud-init status --long
cloud-init schema --system
journalctl -u cloud-init -b
less /var/log/cloud-init.log
less /var/log/cloud-init-output.log
```

Check that the datasource was found, that the YAML is valid and which module failed. To re-test on a throwaway machine, run `cloud-init clean --logs` and reboot. Never do that on a node that already serves traffic.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How cloud-init and first-boot configuration fits together

Part	What it does	What goes wrong
cloud-config	The file starts with #cloud-config and uses modules such as users, packages, write_files and runcmd.	Putting secrets in user data that others can read.
Stages	cloud-init runs in order: local (find the datasource), network (get metadata and user data), config (run configuration modules) and final (package installs and scripts).	Assuming cloud-init runs every boot.
Debugging	Check that the datasource was found, that the YAML is valid and which module failed.	Not validating the YAML schema.

Flow: cloud-config to Stages / Stages to Debugging

Incident: The node with no keys

Incident: *A fresh server booted but nobody could log in.*

```
cloud-init[611]: Failed to fetch metadata: connection timed out
cloud-init[611]: no datasource found
status: error
```

The network interface took longer than usual to come up, so cloud-init timed out and fell back to defaults without the SSH key. The team added a retry setting for the datasource, a post-boot health check that fails the node if cloud-init status is not done, and a rescue procedure through the BMC.

Common mistakes

- Putting secrets in user data that others can read
- Assuming cloud-init runs every boot
- Not validating the YAML schema
- Baking per-server details into the image

Check yourself

What does cloud-init mostly run on?

Answer: The first boot of an instance

What is user data?

Answer: Configuration you supply to the instance

Which command shows cloud-init health?

Answer: cloud-init status --long

Why keep images generic?

Answer: So one image serves many roles

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Firmware and BIOS management at scale

12 minutes. Keep BIOS, BMC, NIC and disk firmware consistent across a fleet with baselines, staged rollouts and drift checks.

Two identical servers can behave differently because of a firmware version or a single BIOS option. At fleet scale you need a baseline, a safe update process and a way to notice drift before users do.

What you will learn

- What firmware lives in a server
- Defining baselines for versions and BIOS settings
- Rolling updates in waves with rollback
- Detecting and fixing drift

What to manage

- **BIOS or UEFI settings:** power profile, hyperthreading, virtualization, boot mode
- **BMC firmware:** the management controller itself
- **NIC, HBA and RAID firmware:** often the cause of odd packet loss or disk resets
- **Drive firmware:** sometimes fixes known bugs or data-loss issues

Baseline and inventory

```
# baseline for one hardware model
model: dl-gen11
bios:
  version: "2.14.1"
  settings:
    PowerProfile: MaxPerformance
    Virtualization: Enabled
    BootMode: Uefi
bmc: "3.20"
nic:
  version: "22.31.6"
```

Read current values through Redfish, for example the `Bios` and `UpdateService` resources, or the vendor tool, and compare to the baseline. Store results with the server's asset record.

```
curl -sk -u "admin:$BMC_PASS" https://10.20.0.15/redfish/v1/Systems/1/Bios | jq '.Attributes.Power \
    Profile'
curl -sk -u "admin:$BMC_PASS" https://10.20.0.15/redfish/v1/UpdateService/FirmwareInventory | jq ' \
    .Members|length'
```

Rolling out

Update one **canary** node per model, run burn-in and workload tests, then expand in small waves while watching error rates. Never update both members of a redundant pair at the same time, and drain workloads first. Keep the previous firmware image available, because some updates cannot be reverted and a rollback plan may mean replacing a part.

Drift

Run a scheduled job that compares live values with the baseline and opens a ticket on mismatch. After repairs or part swaps, re-apply the baseline before the node rejoins the pool.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Firmware and BIOS management at scale fits together

Part	What it does	What goes wrong
What to manage	BIOS or UEFI settings: power profile, hyperthreading, virtualization, boot mode.	Skipping BIOS settings in the baseline.
Baseline and inventory	Read current values through Redfish, for example the Bios and UpdateService resources, or the vendor tool, and compare to the baseline.	Failing to re-apply the baseline after repairs.
Rolling out	Update one canary node per model, run burn-in and workload tests, then expand in small waves while watching error rates.	Updating all nodes of a redundant pair together.
Drift	Run a scheduled job that compares live values with the baseline and opens a ticket on mismatch.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What to manage to Baseline and inventory / Baseline and inventory to Rolling out / Rolling out to Drift

Incident: The slow rack

Incident: *A group of nodes ran 30 percent slower after maintenance.*

```
$ lscpu | grep MHz
CPU MHz:  1196.000
BIOS PowerProfile: PowerSaving (baseline: MaxPerformance)
```

A mainboard swap reset the BIOS to defaults and nobody re-applied the baseline. CPUs sat at a low clock, and latency rose. The team added a post-repair step that applies and verifies the baseline and a weekly drift report across the fleet.

Common mistakes

- Updating all nodes of a redundant pair together
- Skipping BIOS settings in the baseline
- Not keeping old firmware images
- Failing to re-apply the baseline after repairs

Check yourself

What belongs in a firmware baseline?

Answer: Versions and key BIOS settings per model

Who should be updated first?

Answer: A canary node

How do you find drift?

Answer: Compare live values with the baseline regularly

Why drain a node before a firmware update?

Answer: Updates may reboot or fail

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. MAAS and Ironic: provisioning bare metal as a service

13 minutes. See how MAAS and OpenStack Ironic discover, enroll, commission and deploy physical servers through an API like cloud instances.

Racking a server is not the end of the work. You must register it, test it, install an operating system and later wipe it. MAAS and Ironic automate that lifecycle so bare metal can be requested like a virtual machine.

What you will learn

- The lifecycle: enroll, commission, allocate, deploy, release
- What commissioning and testing collect
- How deployment uses PXE, images and cloud-init
- Choosing between MAAS and Ironic

Lifecycle

1. **Enroll:** the machine PXE boots or is added with its BMC details.
2. **Commission:** a small ephemeral OS boots, inventories CPU, memory, disks and NICs and runs tests.
3. **Ready and allocate:** the server joins a pool and is reserved by a user or system.
4. **Deploy:** an image is written, networking is configured and cloud-init applies user data.
5. **Release:** the machine is wiped and returned to the pool.

```
maas admin machines read | jq '.[]|{hostname,status_name,power_state}'
maas admin machines allocate cpu_count=32 tags=gpu
maas admin machine deploy "$SYSTEM_ID" distro_series=jammy user_data="$(base64 -w0 cloud-init.yaml \
    )"
```

Ironic

Ironic is the OpenStack bare metal service. It exposes servers as Nova flavors, so users boot a physical machine with the same API as a virtual one. It supports drivers for IPMI and Redfish, deploys over iPXE or virtual media and can perform automated cleaning between tenants.

```
openstack baremetal node list
openstack server create --flavor baremetal-large --image jammy --network prod bm-web-01
```

Choosing

Choose **MAAS** when you want a focused tool for datacenter provisioning with a simple web UI and API. Choose **Ironic** when you already run OpenStack and want bare metal and virtual machines under one control plane. Both rely on a reliable network boot setup and correct BMC credentials.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How MAAS and Ironic: provisioning bare metal as a... fits together

Part	What it does	What goes wrong
Lifecycle	Enroll: the machine PXE boots or is added with its BMC details.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Ironic	Ironic is the OpenStack bare metal service.	Skipping cleaning between users.
Choosing	Choose MAAS when you want a focused tool for datacenter provisioning with a simple web UI and API.	Weak BMC credentials in the controller.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Lifecycle to Ironic / Ironic to Choosing

Incident: The server that kept the last tenant's data

Incident: *A released machine was reallocated without cleaning.*

```
$ lsblk
sda  1.8T  part /data/customer-a
released node 17 reused for tenant B without clean
```

Cleaning was disabled to make release faster. The next tenant could read old data from the disk. The team enabled automated cleaning, verified it with a check after release and added a policy that a node may not return to the pool until wipe and firmware checks pass.

Common mistakes

- Skipping cleaning between users
- Not testing hardware during commissioning
- Weak BMC credentials in the controller
- Relying on manual DHCP changes

Check yourself

What happens during commissioning?

Answer: The machine is inventoried and tested

Which service exposes bare metal in OpenStack?

Answer: Ironic

Why clean disks on release?

Answer: To protect data between users

What often starts a deploy?

Answer: PXE or virtual media boot

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Hardware failure diagnosis and RMA

12 minutes. Diagnose memory, disk, CPU and power faults from logs and sensors, and run a clean return-merchandise process with the vendor.

Most hardware failures leave clues before the outage. The skill is knowing where to look, which counters matter and how to hand the vendor enough proof to approve a replacement quickly.

What you will learn

- Reading the system event log, ECC counters and SMART data
- Telling memory, disk, CPU and power problems apart
- Collecting evidence for an RMA
- Verifying and recording the repair

Where the clues are

```
ipmitool sel elist | tail -5
edac-util -v
dmesg -T | grep -iE "mce|edac|machine check|i/o error"
smartctl -a /dev/sda | grep -E "Reallocated|Pending|Uncorrectable|Power_On"
sensors
```

- **Memory:** corrected ECC errors that keep rising on one DIMM, then uncorrected errors and machine check events
- **Disks:** reallocated or pending sectors, I/O errors, long command timeouts
- **CPU and board:** machine check exceptions, thermal throttling, random reboots
- **Power:** PSU failure events, voltage warnings, losing redundancy

Diagnosis method

Drain the workload first. Read the SEL for the first error, not just the last, and note the slot or serial. Run the vendor's diagnostic or a memory test to reproduce the fault, and swap the suspected part only after the evidence points at it. Changing many parts at once hides the cause.

RMA

Vendors want the serial number, the diagnostic output, the SEL and the exact error. Attach them to one ticket, request an advance replacement when downtime matters and label the failed part for return. After the swap, run burn-in, re-apply the firmware and BIOS baseline, and record the part serial against the server.

```
smartctl -x /dev/sda > /var/tmp/sda-smart.txt
ipmitool sel list > /var/tmp/sel.txt
dmidecode -t memory | grep -E "Locator|Serial|Size"
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Hardware failure diagnosis and RMA fits together

Part	What it does	What goes wrong
Where the clues are	Memory: corrected ECC errors that keep rising on one DIMM, then uncorrected errors and machine check events.	Ignoring corrected errors.
Diagnosis method	Drain the workload first. Read the SEL for the first error, not just the last, and note the slot or serial.	Replacing several parts at once.
RMA	Vendors want the serial number, the diagnostic output, the SEL and the exact error.	Skipping burn-in after a repair.

Flow: Where the clues are to Diagnosis method / Diagnosis method to RMA

Incident: The weekly crash

Incident: *A server crashed once a week until one DIMM was replaced.*

```
EDAC MC0: 1 CE memory read error on CPU_SrcID#0_Channel#2_DIMM#1
EDAC MC0: 14 CE memory read errors on the same DIMM
mce: [Hardware Error]: uncorrected memory error
```

Nobody watched corrected error counts. They grew for weeks and then became an uncorrectable fault that crashed the kernel. After the swap the team added an alert on corrected errors per DIMM and a policy to replace modules before uncorrected errors appear.

Common mistakes

- Replacing several parts at once
- Ignoring corrected errors
- Sending the vendor vague descriptions
- Skipping burn-in after a repair

Check yourself

What do rising corrected ECC errors suggest?

Answer: A DIMM that may fail soon

What should you do before diagnosing a node in service?

Answer: Drain the workload

What helps a vendor approve an RMA?

Answer: Serial numbers and diagnostic output

Why change one part at a time?

Answer: It shows which part caused the fault

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Power, cooling and rack capacity planning

12 minutes. Plan rack power in watts and amps, understand redundant feeds and cooling limits, and place servers without tripping breakers.

You can fill a rack with servers and still be unable to power or cool them. Good operators plan in kilowatts and airflow, keep redundancy honest and watch the real numbers from the power distribution units.

What you will learn

- Reading watts, amps and power factor for a rack
- Redundant A and B feeds and the 80 percent rule
- Airflow, inlet temperature and hot spots
- Tracking stranded and available capacity

Power budget

Estimate from **measured** draw at typical and peak load, not from the label on the power supply. Convert: watts divided by voltage gives amps, so a 7 kW rack on 230 V draws about 30 A. Breakers should not run continuously above about 80 percent of their rating.

```
# read power from a server BMC
ipmitool dcmi power reading
# many PDUs expose SNMP or Redfish; example SNMP read
snmpget -v2c -c public pdu-a-12 .1.3.6.1.4.1.318.1.1.12.1.16.0
```

Redundant feeds

With **A and B** feeds, each server draws from both. If one feed fails, the other carries everything, so the **sum of both** must fit within one feed. That means total load must stay under roughly 40 percent of the combined rating, or under 80 percent of one feed. Balance servers across phases so no single phase overloads.

Cooling

Servers pull cold air in at the front and exhaust hot air at the back. Keep cold and hot aisles separate, use blanking panels in empty slots and watch **inlet temperature** sensors on servers. High inlet temperature means recirculation or too much heat in one place. Dense GPU racks may need liquid cooling or fewer servers per rack.

Capacity tracking

Record per rack: rated power, measured peak, space left, cooling limit and network ports. The smallest of these is the real capacity. Unused space with no spare power is **stranded**. Review monthly and before every order.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Power, cooling and rack capacity planning fits together

Part	What it does	What goes wrong
Power budget	Estimate from measured draw at typical and peak load, not from the label on the power supply.	Budgeting from nameplate instead of measured draw.
Redundant feeds	With A and B feeds, each server draws from both.	Letting A and B feeds both run near their limit.
Cooling	Servers pull cold air in at the front and exhaust hot air at the back.	Leaving gaps in racks without blanking panels.
Capacity tracking	Record per rack: rated power, measured peak, space left, cooling limit and network ports.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Power budget to Redundant feeds / Redundant feeds to Cooling / Cooling to Capacity tracking

Incident: One feed fails, the other trips

Incident: *Losing feed A overloaded feed B and a breaker opened.*

```
PDU-A: input lost
PDU-B: load 31.4A of 32A rated
PDU-B: breaker tripped, 24 servers down
```

The rack had been designed as redundant, but both feeds already ran near capacity. When A failed, B took the full load and tripped. After the outage the team set a rule that the combined load must fit on one feed with headroom, and added alerts at 70 percent per feed.

Common mistakes

- Budgeting from nameplate instead of measured draw
- Letting A and B feeds both run near their limit
- Leaving gaps in racks without blanking panels
- Ignoring inlet temperature trends

Check yourself

What does N+1 power redundancy require?

Answer: Remaining feeds can carry the full load

What does a high inlet temperature suggest?

Answer: Poor airflow or too much heat nearby

Why use blanking panels?

Answer: They stop hot air flowing back to the front

What is stranded capacity?

Answer: Space or ports unusable because power or cooling is exhausted

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Bare metal and datacenter

Remote management, provisioning, disks and RAID, bonding and the first checks on a sick server.

Talk to the BMC

```
ipmitool -H BMC_IP -U USER -P PASS chassis power status
ipmitool -H BMC_IP -U USER -P PASS chassis power cycle
ipmitool -H BMC_IP -U USER -P PASS sel elist
ipmitool -H BMC_IP -U USER -P PASS sensor list
```

Warning: `power cycle` *and* `power off` *cut power at once. Drain the host first unless it is already hung.*

Check disks

```
lsblk -f
sudo smartctl -H /dev/sda
sudo smartctl -a /dev/sda | grep -E "Reallocated|Pending|Power_On"
cat /proc/mdstat
sudo mdadm --detail /dev/md0
```

Replace a failed RAID member

```
sudo mdadm /dev/md0 --fail /dev/sdb
sudo mdadm /dev/md0 --remove /dev/sdb
sudo mdadm /dev/md0 --add /dev/sdc
watch cat /proc/mdstat
```

Network and bonding

```
cat /proc/net/bonding/bond0
ip -s link show bond0
ethtool eno1 | grep -E "Speed|Link detected"
dmesg | grep -i bond
```

PXE problems

```
sudo journalctl -u dnsmasq -f
grep -E "dhcp-boot|next-server|filename" /etc/dnsmasq.conf
sudo tcpdump -ni eth0 port 67 or port 68 or port 69
```

A sick server checklist

1. Drain the workloads.
2. Read the BMC event log, SMART, kernel log and switch counters.
3. Decide which part is at fault. Do not swap parts at random.
4. Change one thing, test, record it by asset tag.

Quick inventory

```
lscpu | head -5
free -h
lsblk -o NAME,SIZE,MODEL,SERIAL
dmidecode -s system-serial-number
ip -br link
```

Cautions

Danger: `ipmitool chassis power off` *and* `power cycle` *cut power with no clean shutdown. Drain the workload first, and check you have the right BMC address.*

Warning: *Never run* `mdadm --zero-superblock` *or* `--create` *on a disk that still holds data. Check* `mdadm --detail` *and* `lsblk` *first.*

Warning: *A BMC on a shared or routable network is an entry point to the machine. Keep it on an isolated management network and change default passwords.*

Caution: *Replacing a failed disk starts a rebuild that reads every other disk. Confirm the backup works before you pull the disk.*

MARGA by EknathaLabs
marga.eknathalabs.com